

Programación I

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Alumnos - Grupo n° 72

Héctor Atilio Signoriello - Comisión 10

Gregorio Agustin Hernandez - Comisión 12

Docentes:

Ariel Enferrel

Martín A. García

Cinthia Rigoni

Tutores:

Tomás Ferro

Martina Belen Zabala

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

XX de Junio de 2026

Tabla de contenido

Marco Teórico	3
Decisiones Técnicas y Arquitectura	7
Dificultades y Conclusiones	8
Bibliografía / Webgrafía	9
Links	9

Trabajo Práctico Integrador (TPI)

Marco Teórico:

Durante el desarrollo del trabajo práctico se emplearon diversos conceptos fundamentales del lenguaje Python, ampliamente documentados en la documentación oficial de Python.

- **Listas:** Las listas constituyen una de las estructuras de datos fundamentales de Python. Se trata de colecciones ordenadas, mutables y dinámicas que permiten almacenar elementos de diferentes tipos de datos dentro de una misma estructura. Internamente, Python implementa las listas mediante arreglos dinámicos, lo que permite un acceso eficiente a los elementos mediante índices.

Las listas admiten operaciones de inserción, eliminación, modificación y recorrido. Además, proporcionan métodos integrados para manipular los datos, tales como **append()**, **insert()**, **remove()**, **pop()**, **extend()** y **sort()**. Gracias a estas características, resultan especialmente útiles para almacenar registros provenientes de archivos CSV, representar conjuntos de datos y realizar operaciones de análisis sobre ellos.

En el contexto del presente trabajo práctico, las listas fueron utilizadas para almacenar los registros de ventas extraídos del archivo CSV, permitiendo posteriormente realizar recorridos secuenciales para calcular métricas estadísticas y generar reportes.

- **Diccionarios:** Los diccionarios son estructuras de datos basadas en tablas hash que almacenan información mediante pares clave-valor. Cada clave debe ser única e inmutable, mientras que los valores asociados pueden pertenecer a cualquier tipo de dato.

La implementación basada en hashing permite que las operaciones de búsqueda, inserción y actualización presenten una complejidad temporal promedio de $O(1)$, lo que los convierte en una estructura altamente eficiente para el acceso a información.

Los diccionarios son ampliamente utilizados para representar registros estructurados, configuraciones y agrupaciones de datos. En proyectos de análisis de datos, permiten asociar atributos específicos a cada registro de manera clara y eficiente.

Durante el desarrollo del trabajo práctico, los diccionarios se emplearon para representar cada venta individual, utilizando como claves los nombres de los campos del archivo CSV (**id**, **sale_date** y **sale_amount**). Esta representación facilitó el acceso a los datos mediante nombres descriptivos en lugar de índices numéricos.

- **Funciones:** Las funciones son bloques de código reutilizables que encapsulan una determinada tarea o conjunto de instrucciones. Constituyen uno de los principios fundamentales de la programación modular, ya que permiten dividir un problema complejo en subproblemas más pequeños y manejables.

Una función puede recibir parámetros de entrada, procesarlos y devolver uno o más resultados mediante la instrucción **return**. Este mecanismo favorece la reutilización del código, reduce la duplicación de instrucciones y mejora la mantenibilidad de los programas.

Desde el punto de vista del diseño de software, las funciones promueven los principios de cohesión y separación de responsabilidades, permitiendo que cada componente del sistema se encargue de una única tarea específica.

En este trabajo práctico se implementaron funciones para la carga de datos, procesamiento de registros, cálculo de estadísticas y generación de resultados, logrando una estructura de código más organizada y fácil de mantener.

- **Estructuras Condicionales:** Las estructuras condicionales permiten modificar el flujo de ejecución de un programa en función de la evaluación de expresiones lógicas. Python proporciona las instrucciones **if**, **elif** y **else** para implementar mecanismos de toma de decisiones.

El funcionamiento de una estructura condicional se basa en la evaluación de expresiones booleanas que pueden resultar verdaderas (True) o falsas (False). Dependiendo del resultado obtenido, el programa ejecutará diferentes bloques de código.

Las estructuras condicionales son fundamentales para la validación de datos, el control de errores y la aplicación de reglas de negocio. También permiten implementar filtros sobre conjuntos de datos y seleccionar acciones específicas según determinadas condiciones.

En el análisis realizado, las estructuras condicionales fueron utilizadas para validar registros, identificar valores extremos y clasificar datos según criterios previamente definidos.

- **Ordenamiento de Datos:** El ordenamiento consiste en reorganizar los elementos de una colección siguiendo un criterio específico. Python proporciona dos mecanismos principales para esta tarea: la función **sorted()** y el método **sort()** de las listas.

El método **sort()** modifica directamente la lista original, mientras que **sorted()** genera una nueva colección ordenada sin alterar la estructura original. Ambos mecanismos utilizan internamente el algoritmo **Timsort**, una combinación optimizada de **Merge Sort** e **Insertion Sort** desarrollada específicamente para Python.

Timsort posee una complejidad temporal de **$O(n \log n)$** en el caso promedio y presenta un excelente rendimiento sobre conjuntos de datos parcialmente ordenados, situación frecuente en aplicaciones reales.

El ordenamiento de datos resulta esencial para tareas de análisis, generación de reportes y visualización de información. En este proyecto fue utilizado para organizar registros de ventas según criterios como fechas o montos de venta, facilitando la interpretación de los resultados obtenidos.

- **Estadísticas Básicas:** La estadística descriptiva proporciona herramientas matemáticas para resumir y analizar conjuntos de datos. Entre las medidas más utilizadas se encuentran:
 - **Suma:** total acumulado de los valores analizados.
 - **Promedio (media aritmética):** valor representativo obtenido al dividir la suma de los elementos por la cantidad total de observaciones.
 - **Máximo:** mayor valor presente en el conjunto de datos.
 - **Mínimo:** menor valor presente en el conjunto de datos.
 - **Conteo:** cantidad total de registros analizados.

Estas métricas permiten identificar tendencias generales, detectar anomalías y evaluar el comportamiento de los datos.

En el presente trabajo práctico se calcularon estadísticas descriptivas sobre los montos de ventas registrados en el archivo CSV, permitiendo obtener una visión general del desempeño comercial representado por los datos simulados.

- **Archivos CSV:** El formato CSV (Comma-Separated Values) constituye uno de los estándares más utilizados para el almacenamiento e intercambio de datos tabulares debido a su simplicidad y compatibilidad entre sistemas.

Cada fila del archivo representa un registro, mientras que las columnas contienen atributos específicos separados mediante delimitadores. Python proporciona el módulo estándar `csv`, que incluye herramientas para la lectura y escritura de archivos CSV de forma eficiente.

Las clases **`csv.reader`** y **`csv.DictReader`** permiten transformar el contenido de un archivo CSV en estructuras de datos que pueden ser procesadas por el programa. De manera similar, **`csv.writer`** y **`csv.DictWriter`** facilitan la generación de nuevos archivos a partir de los resultados obtenidos.

En este proyecto, el archivo CSV funcionó como fuente principal de datos. Los registros fueron cargados en memoria, procesados mediante estructuras de datos de Python y posteriormente utilizados para generar estadísticas y reportes derivados del análisis realizado.

Decisiones Técnicas y Arquitectura

El sistema fue diseñado siguiendo el principio de modularización, dividiendo el código en **módulos** independientes según su responsabilidad:

manejo_dataset.py para la carga, agregado y actualización del archivo CSV; **busqueda.py** para las búsquedas por nombre; **filtros.py** para el filtrado de datos; **ordenamiento.py** para los ordenamientos; **estadisticas.py** para el cálculo de indicadores; y **utilidades.py** para funciones auxiliares compartidas como la impresión y validación de entradas.

Como estructura principal de datos se utilizó una lista de diccionarios, dado que el módulo **csv.DictReader** de Python transforma automáticamente cada fila del archivo CSV en un diccionario, asociando cada valor a su clave correspondiente según el encabezado del archivo. Esta estructura resultó conveniente para acceder a los datos de cada país mediante nombres descriptivos en lugar de índices numéricos.

Al iniciar el programa, el archivo CSV es cargado completamente en memoria, y todas las operaciones se realizan sobre esa estructura. En los casos donde se modifica el dataset, el archivo CSV es reescrito en su totalidad para mantener la persistencia de los datos.

El manejo de errores fue implementado mediante bloques **try/except** tanto en las funciones de cada módulo como en el menú principal, garantizando que el programa no se interrumpa ante entradas inválidas del usuario.

Dificultades y Conclusiones

Durante el desarrollo del proyecto no se presentaron dificultades técnicas significativas. El principal aprendizaje fue la correcta aplicación de los conceptos de modularización y estructuras de datos trabajados durante la cursada, llevándolos a la práctica en un sistema funcional completo.

El trabajo en equipo mediante Git y el uso de ramas para desarrollar cada funcionalidad de forma independiente resultó una experiencia positiva, permitiendo organizar el trabajo de manera ordenada y colaborativa.

Como conclusión, el desarrollo del presente trabajo permitió afianzar el uso de listas, diccionarios, funciones, estructuras condicionales y archivos CSV en Python, comprendiendo la importancia de escribir código claro, comentado y modularizado.

Bibliografía / Webgrafía

Python Software Foundation. (2025). Built-in types. Python Documentation.

<https://docs.python.org/es/3.14/library/stdtypes.html>

Python Software Foundation. (2025). Data structures. Python Documentation.

<https://docs.python.org/es/3.14/tutorial/datastructures.html>

Python Software Foundation. (2025). Control flow tools. Python Documentation.

<https://docs.python.org/es/3.14/tutorial/controlflow.html>

Python Software Foundation. (2025). CSV file reading and writing. Python Documentation.

<https://docs.python.org/es/3.14/library/csv.html>

Python Software Foundation. (2025). Functions. Python Documentation.

<https://docs.python.org/es/3.14/tutorial/controlflow.html#defining-functions>

Links:

Hernandez Gregorio Agustin, Signoriello Héctor Atilio (2026). Sistema de Gestión de Datos de Países [Código fuente]. GitHub.

Repositorio del proyecto <https://github.com/signoriellohector/dataset-de-paises>

Hernandez Gregorio Agustin, Signoriello Héctor Atilio (2026). Sistema de Gestión de Datos de Países [Vídeo]. Youtube.

Video del proyecto <https://www.youtube.com/watch?v=-Su3KfID2As>